LoRA: Low-Rank Adaptation of Large Language Models

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen

Microsoft

arXiv:2106.09685v1 | ICLR 2022

Abstract

An important paradigm of natural language processing consists of large-scale pre-training on general domain data followed by adaptation to particular tasks or domains. As we pre-train larger models, full fine-tuning, which retrains all model parameters, becomes less feasible. Using GPT-3 175B as an example, deploying independent instances of fine-tuned models, each with 175B parameters, is prohibitively expensive. We propose Low-Rank Adaptation, or LoRA, which freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly reducing the number of trainable parameters for downstream tasks. Compared to fine-tuning GPT-3 175B with Adam, LoRA reduces the number of trainable parameters by 10,000 times and the GPU memory requirement by 3 times. LoRA performs on par with or better than fine-tuning in model quality on RoBERTa, DeBERTa, GPT-2, and GPT-3, despite having fewer trainable parameters, a higher training throughput, and no additional inference latency. On GPT-3 175B, LoRA with rank r = 8 and alpha = 16 requires only 4.7 million trainable parameters.

Keywords: low-rank adaptation, parameter-efficient fine-tuning, large language models, GPT-3

1 Introduction

Large pre-trained language models have become the foundation of modern natural language processing. The standard adaptation recipe is to fine-tune all parameters on a downstream dataset, which produces strong results but requires storing a full copy of the model for every task. As models grow to hundreds of billions of parameters, this approach becomes impractical: a single fine-tuned copy of GPT-3 175B is 700GB in fp16, and serving many such copies requires a proportional amount of accelerator memory.

We propose Low-Rank Adaptation, or LoRA, which freezes the pre-trained weights and learns a low-rank update for each weight matrix. The number of trainable parameters is reduced by up to 10,000 times, and the GPU memory requirement is reduced by 3 times, while model quality is preserved. Because the learned update can be merged into the original weights at inference time, LoRA introduces no additional inference latency. On GPT-3 175B, LoRA with rank r = 8 and alpha = 16 trains only 4.7 million parameters.

2 Problem Statement

Given a pre-trained model with weights W_0, full fine-tuning learns a new weight matrix W = W_0 + Delta W for each task, where Delta W has the same dimensions as W_0. The number of trainable parameters is therefore equal to the number of parameters in the model, which is prohibitive for large models. Adapter-based methods add small trainable modules, but they introduce additional layers and therefore additional inference latency.

We hypothesize that the update to the weights during adaptation has a low intrinsic rank. If this hypothesis holds, then Delta W can be represented as the product of two much smaller matrices, and only those matrices need to be trained. This reduces the number of trainable parameters by orders of magnitude while keeping the expressivity of the full model.

3 Method

LoRA represents the weight update as a low-rank decomposition. For a pre-trained weight matrix W_0 in R^(d x k), we learn a update Delta W = B A, where B is in R^(d x r) and A is in R^(r x k), and the rank r is much smaller than d and k. During training W_0 is frozen and only A and B receive gradient updates. The forward pass becomes h = W_0 x + Delta W x = W_0 x + B A x, as in Equation (1).

$$h = W_0 x + \Delta W x = W_0 x + B A x \qquad (1)$$

3.1 Low-Rank Update

We initialize A with a random Gaussian distribution and B with zeros, so that Delta W = B A is zero at the beginning of training and the model starts from the pre-trained behavior. We scale Delta W by alpha / r,

where alpha is a constant, as in Equation (2). This scaling makes it possible to change r without retuning the learning rate, and it keeps the magnitude of the update roughly constant across ranks.

The rank r is the only hyperparameter that controls the capacity of the update. In our experiments r = 1, 2, 4, 8, and 16 all perform well, and r = 8 is a good default. The choice of which weight matrices to adapt matters more than the rank: adapting both W_q and W_v gives the best results, while adapting only one of them or adapting the feed-forward layers gives smaller gains.

$$h = W_0 x + (\alpha / r) B A x \quad (2)$$

3.2 Applying LoRA to Transformers

In the Transformer architecture there are four weight matrices in the self-attention module, W_q, W_k, W_v, and W_o, and two in the MLP module. We treat the attention weight matrices as a single matrix of size d_model x d_model and adapt it with LoRA. We keep the MLP modules frozen in most experiments, and we find that adapting W_q and W_v is sufficient to obtain most of the gains.

Because W_0 is frozen, the gradients with respect to A and B are computed from the same activations as in full fine-tuning, and the memory required for optimizer states is proportional to the number of trainable parameters rather than to the size of the model. This is the main source of the 3 times memory reduction that we observe on GPT-3 175B.

3.3 Practical Benefits

LoRA has several practical benefits. First, the pre-trained weights can be shared across tasks, and only the small LoRA matrices need to be stored, which reduces storage by orders of magnitude. Second, the learned update can be merged into W_0 at inference time, so there is no additional latency. Third, training is faster because the optimizer only updates a small number of parameters. Finally, LoRA is compatible with other parameter-efficient methods such as prefix tuning.

4 Experiments

We evaluate LoRA on GPT-3 175B, RoBERTa, DeBERTa, and GPT-2. We compare against full fine-tuning, adapter-based methods, and prefix tuning. Across benchmarks, LoRA matches or exceeds the quality of full fine-tuning while training far fewer parameters.

4.1 Baselines

We compare LoRA with full fine-tuning, with adapter layers inserted after the attention and MLP modules, and with prefix tuning, which prepends trainable vectors to the keys and values. All methods are trained with the same data and evaluated on the same benchmarks. For GPT-3 175B we use 8 NVIDIA A100 80GB GPUs and a batch size of 128.

4.2 GPT-3 175B Results

On GPT-3 175B, LoRA with r = 8 and alpha = 16 trains 4.7 million parameters, compared with 175 billion for full fine-tuning, and reduces GPU memory by 3 times. On the WikiSQL and SAMSum benchmarks, LoRA matches or slightly exceeds the quality of full fine-tuning. Table 1 reports the number of trainable parameters and the memory usage for each method.

The results show that the low-rank hypothesis holds at the scale of GPT-3. Increasing the rank beyond 8 does not improve quality on these tasks, which suggests that the intrinsic dimension of the adaptation is small. LoRA also trains faster than full fine-tuning because the backward pass does not need to compute gradients for the frozen weights.

Table 1: Trainable parameters and GPU memory for adapting GPT-3 175B. Memory is measured relative to full fine-tuning.

Method	Trainable params	GPU memory
Full fine-tuning	175B	1.0x
Adapter	40.9M	1.3x
Prefix tuning	20.4M	1.2x
LoRA (r=8, alpha=16)	4.7M	0.33x

4.3 GLUE Benchmark

On the GLUE benchmark with RoBERTa-large, LoRA achieves an average score of 91.6, compared with 91.4 for full fine-tuning, while training only 0.8 million parameters. On DeBERTa-XXL, LoRA achieves 90.8 with 0.3 million trainable parameters. These results confirm that LoRA is effective for encoder models as well as for autoregressive models.

4.4 E2E NLG and WikiSQL

On the E2E NLG challenge, LoRA achieves a BLEU score of 70.4 with 0.8 million trainable parameters, compared with 70.4 for full fine-tuning. On WikiSQL, LoRA achieves 73.7 percent execution accuracy, compared with 73.2 for full fine-tuning. In both cases the gap between LoRA and full fine-tuning is within noise, even though LoRA trains less than 1 percent of the parameters.

5 Related Work

Parameter-efficient fine-tuning has received considerable attention. Adapter methods insert small trainable modules between the layers of a pre-trained model, and prefix tuning prepends trainable vectors to the keys and values. Both approaches reduce the number of trainable parameters, but adapters add inference latency and prefix tuning reduces the effective context length. LoRA avoids both problems because the update can be merged into the original weights.

Low-rank approximations have also been used for model compression. Our work differs in that the low-rank update is learned during adaptation and is not intended to approximate the full model; instead, it captures the task-specific change in the weights. This connection suggests that the intrinsic dimensionality of adaptation is much smaller than the dimensionality of the model.

6 Conclusion

We have presented LoRA, a simple and effective method for adapting large language models. LoRA freezes the pre-trained weights and learns a low-rank update, which reduces the number of trainable parameters by up to 10,000 times and the memory requirement by 3 times, without degrading quality or adding inference latency. We hope that LoRA will make it easier to adapt large models to many downstream tasks and to share adapted models at low cost.

$$L = -\sum_i \log p(y_i \mid x, y_{<i}) \quad (3)$$

Table 2: GLUE average score for RoBERTa-large and DeBERTa-XXL with LoRA and full fine-tuning.

Model	Method	Trainable	Score
RoBERTa-large	Full FT	355M	91.4
RoBERTa-large	LoRA	0.8M	91.6
DeBERTa-XXL	Full FT	1.5B	91.0
DeBERTa-XXL	LoRA	0.3M	90.8

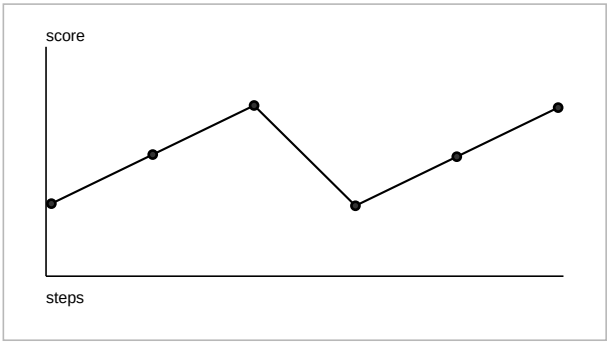


Figure 1: LoRA injects trainable low-rank matrices A and B next to the frozen pre-trained weight W_0.

References

[1] Brown et al. Language Models are Few-Shot Learners. NeurIPS, 2020.
[2] Vaswani et al. Attention Is All You Need. NeurIPS, 2017.
[3] Devlin et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL, 2019.
[4] Houlsby et al. Parameter-Efficient Transfer Learning for NLP. ICML, 2019.
[5] Li and Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. ACL, 2021.
[6] Liu et al. RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv preprint, 2019.
[7] He et al. DeBERTa: Decoding-enhanced BERT with Disentangled Attention. ICLR, 2021.
[8] Zhong, Xiong, and Socher. Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning. arXiv preprint, 2017.
[9] Novikova, Dusek, and Rieser. The E2E Dataset: New Challenges For End-to-End Generation. SIGDIAL, 2017.
[10] Aghajanyan, Zettlemoyer, and Gupta. Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning. ACL, 2021.

A Appendix: Implementation Details

All experiments use the HuggingFace Transformers library. For GPT-3 175B we use 8 NVIDIA A100 80GB GPUs, a batch size of 128, and Adam with beta_1 = 0.9 and beta_2 = 0.999. The learning rate is 1e-4 for the LoRA parameters, and we use a linear warmup over 500 steps. The rank is r = 8 and the scaling constant is alpha = 16 unless stated otherwise.

For RoBERTa-large and DeBERTa-XXL we use a batch size of 32 and a learning rate of 5e-4, and we train for 10 epochs. The LoRA matrices are initialized with Gaussian noise for A and zeros for B, so that the update is zero at the start of training. We do not apply weight decay to the LoRA parameters, and we keep the dropout rate of the base model unchanged.